

Casamento de Padrão

Prof. Diego Silva Caldeira Rocha

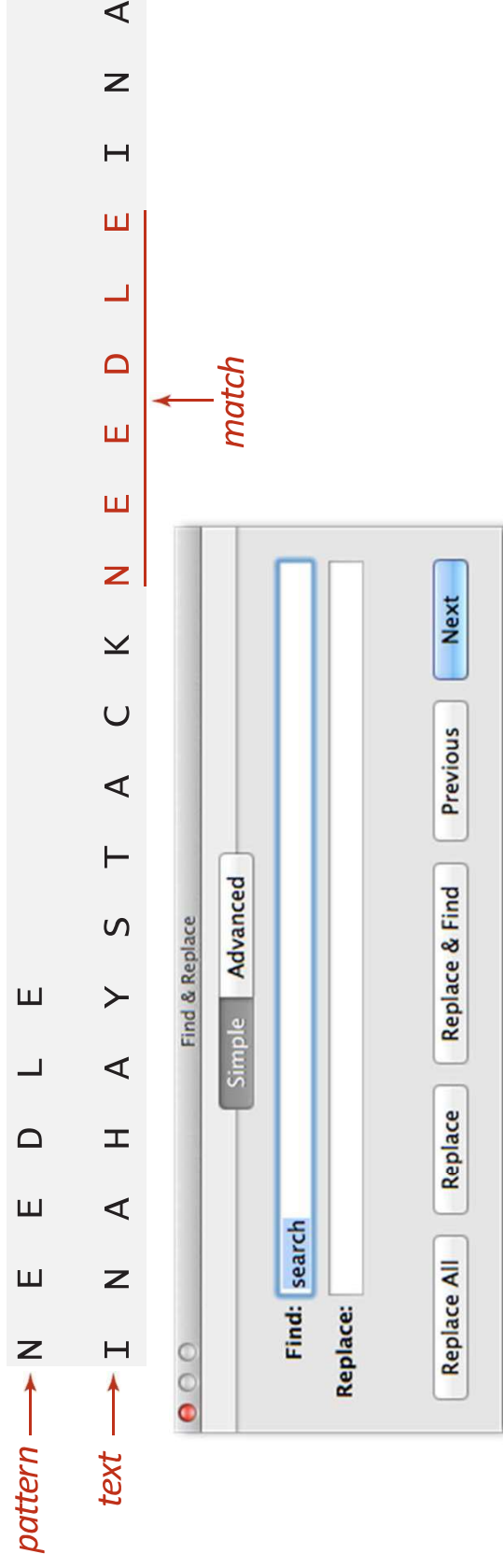
Sumário

- Introdução
- Força Bruta
- Knuth-Morris-Pratt
- Boyer-Moore
- Rabin-Karp

Casamento de Padrão

Objetivo. Encontrar M em texto de comprimento N.

tipicamente $N \gg M$



Exemplo de Aplicação

Perícia em Computador. Pesquise na memória ou no disco por assinaturas, por exemplo, todas as URLs ou chaves RSA que o usuário inseriu.

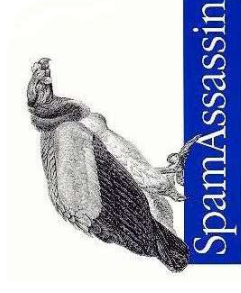


<http://citp.princeton.edu/memory>

Exemplo de Aplicação

Identificar padrões indicativos de spam.

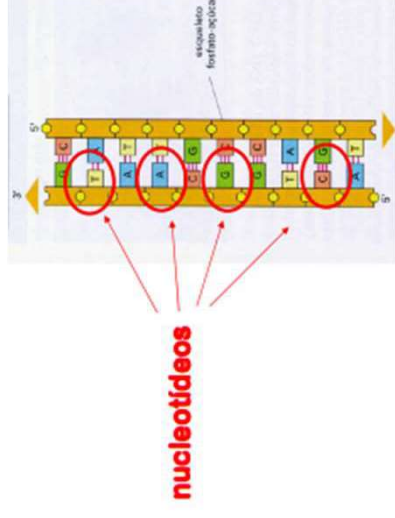
- Lucros
- Perder Peso
- Viagra à base de Ervas
- Não há pegadinhas.
- Este é um envio único.
- Esta mensagem é enviada em conformidade com as normas de spam.



Exemplo de Aplicação

Identificar padrões em DNA.

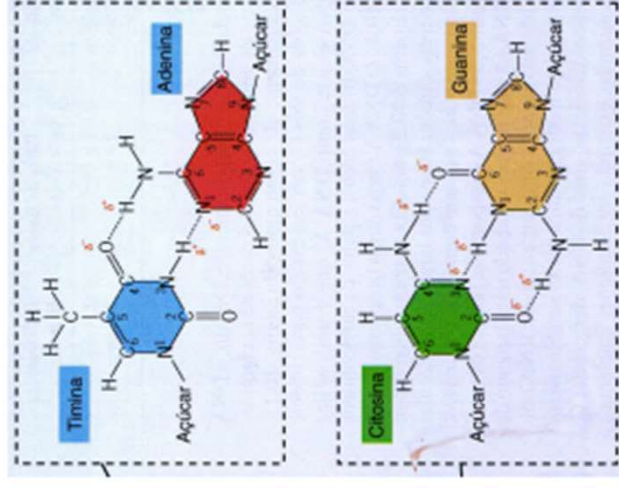
- Teste de Paternidade e Parentesco
- Investigação Forense
- Diagnóstico de Doenças Genéticas
- Genealogia e Ancestralidade
- Pesquisa Científica



Nucleotídeos

Adenina
Timina
Guanina
Citosina

bases nitrogenadas



Exemplo de Aplicações

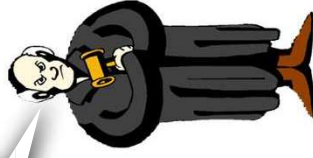
Vigilância Eletrônica.



É necessário
monitorar todo o
tráfego da internet.
(Segurança)



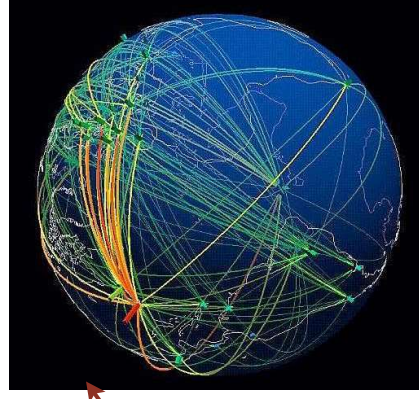
De jeito
nenhum
(privacidade)



Bem, estamos principalmente
interessados em
“ATAQUE AO AMANHECER”



OK. Construa
uma máquina que só
procure por isso.



“ATAQUE AO
AMANHECER”
encontrado

Exemplo de Aplicações

Screen scraping. Extraia dados relevantes de uma página web.

Ex. Encontre uma string delimitada por and após a primeira ocorrência do padrão: Last Trade:.

```
...
<tr>
<td class= "yfnc_tablehead1"
width= "48%">
Last Trade:
</td>
<td class= "yfnc_tabledata1">
<big><b>452.92</b></big>
</td></tr>
<td class= "yfnc_tablehead1"
width= "48%">
Trade Time:
</td>
<td class= "yfnc_tabledata1">
...
```

Google Inc. (NasdaqGS: GOOG)

After Hours: 0.00 N/A (N/A) 10:00PM EST

582.93

Nov 29

0.00 (0.00%)

582.93

N/A

579.70 x 100

585.33 x 100

731.10

Day's Range:

52wk Range:

Volume:

Avg Vol (3m):

Market Cap:

P/E (ttm):

EPS (ttm):

Div & Yield:

N/A - N/A

473.02 - 642.96

0

3,100,480

188.80B

19.87

29.34

N/A (N/A)

Google Inc.

GOOG



1d 5d 1y 2y 5y max

<http://finance.yahoo.com/q?s=goog>

Screen scraping: Implementação em Java

Java library. O método `indexOf()` na biblioteca de strings do Java retorna o índice da primeira ocorrência de uma string dada, começando em um deslocamento dado.

```
public class StockQuote
{
    public static void main(String[] args)
    {
        String name = "
```

Força bruta substring search

Verifique o padrão começando em cada posição do texto.

i	j	i+j	<u>txt</u> → A B A C A C A D A B R A C									
0	2	2	A	B	R	A	pat					
1	0	1	A	B	R	A						
2	1	3	A	B	R	A						
3	0	3				A	B	R	A			
4	1	5					A	B	R	A		
5	2	7										
6	4	10										

entries in black match the text

entries in gray are for reference only

return i when j is M

match

Força bruta substring search: Java Implementação

Verifique o padrão começando em cada posição do texto.

i	j	i+j	0	1	2	3	4	5	6	7	8	9	10
			A	B	A	C	A	D	A	B	R	A	C
4	3	7					A	D	A	C	R		
5	0	5					A	D	A	C	R		

```
public static int search(String pat, String txt)
{
    int M = pat.length();
    int N = txt.length();
    for (int i = 0; i <= N - M; i++)
    {
        int j;
        for (j = 0; j < M; j++)
            if (txt.charAt(i+j) != pat.charAt(j))
                break;
        if (j == M) return i;
    }
    return N;
}
```

index in text where pattern starts

not found

Pesquisa de substring de força bruta: pior caso

O algoritmo de força bruta pode ser lento se o texto e o padrão forem repetitivos.

i	j	i+j	0	1	2	3	4	5	6	7	8	9
			txt → A A A A A A A A A A B									
0	4	4	A	A	A	A	A	B	← pat			
1	4	5		A	A	A	A	B				
2	4	6			A	A	A	A	B			
3	4	7				A	A	A	A	B		
4	4	8					A	A	A	A	B	
5	5	10						A	A	A	A	B

↑
match

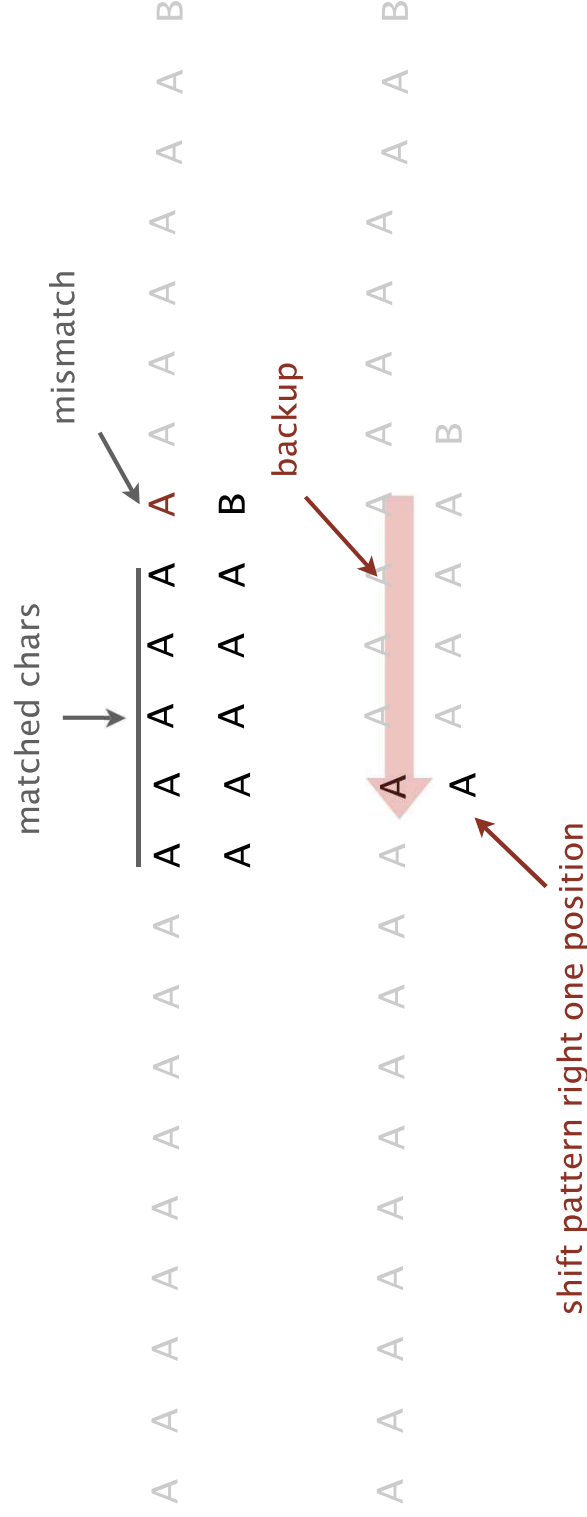
Pior Caso. ~ MN comparação de caracteres.

Backup

Em muitas aplicações, queremos evitar o backup no fluxo de texto.

- Trate a entrada como um fluxo de dados
- Modelo abstrato: entrada padrão

O algoritmo de força bruta precisa de backup para cada incompatibilidade.



Abordagem 1. Manter buffer dos últimos M caracteres.

Abordagem 2.

Força-bruta substring search: alternativa de implementação

A mesma sequência de char é comparada à implementação anterior.

- O índice *i* indica o fim da sequência de caracteres já correspondidos no texto.
- *j* armazena o número de caracteres já correspondidos (fim da sequência no padrão).

<i>i</i>	<i>j</i>	0	1	2	3	4	5	6	7	8	9	10
		A	B	A	C	A	D	A	B	R	A	C
7	3				A	D	A	C	R			
5	0					A	D	A	C	R		

```
public static int search(String pat, String txt)
{
    int i, N = txt.length();
    int j, M = pat.length();
    for (i = 0, j = 0; i < N && j < M; i++)
    {
        if (txt.charAt(i) == pat.charAt(j)) j++;
        else { i -= j; j = 0; }
    }
    if (j == M) return i - M;
    else
        return N;
}
```



Desafios algorítmicos na busca de substrings

A força bruta nem sempre é suficiente.

Desafio Teórico. Garantia de tempo linear.

← fundamental algorithmic problem

Desafio Prático. Evite backup em fluxo de texto.

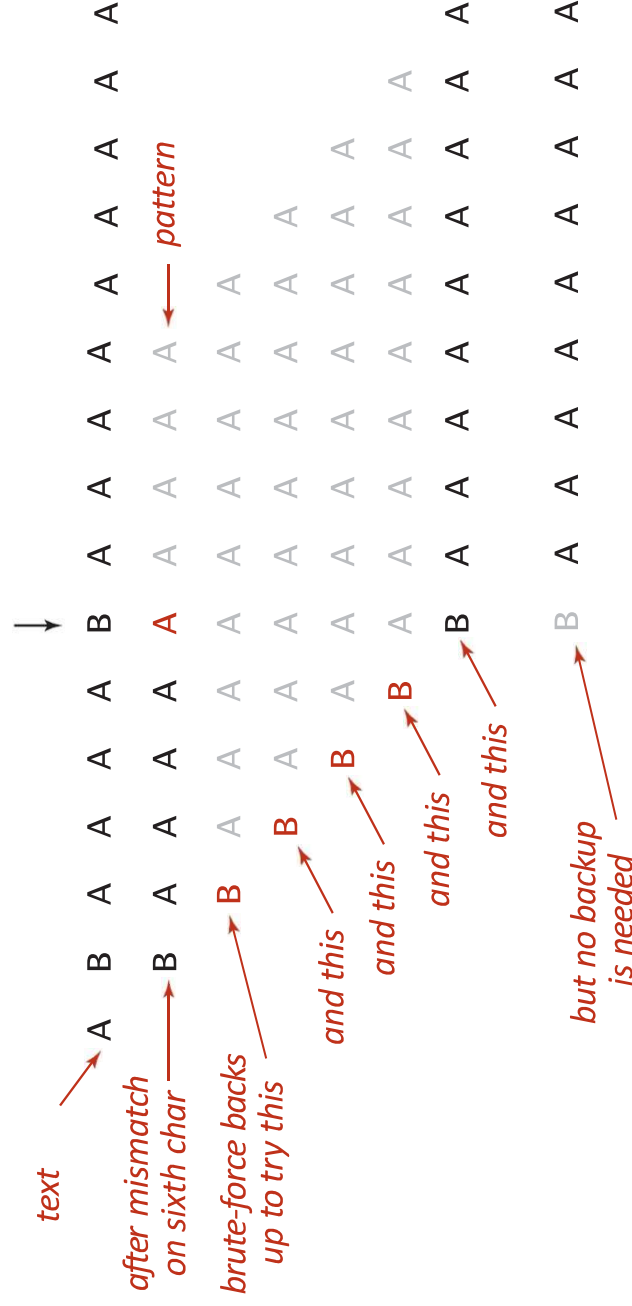
← often no room or time to save text

Now is the time for all people to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for many good people to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for a lot of good people to come to the aid of their party. Now is the time for all of the good people to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for each good person to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for all good Republicans to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for many or all good people to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for all good Democrats to come to the aid of their party. Now is the time for all people to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for many good people to come to the aid of their party. Now is the time for a lot of good people to come to the aid of their party. Now is the time for all of the good people to come to the aid of their party. Now is the time for each person to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for all good Republicans to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for many or all good people to come to the aid of their party. Now is the time for all good people to come to the aid of their party. Now is the time for all good Democrats to come to the aid of their party.

Knuth-Morris-Pratt busca substring

Visão. Suponha que estamos procurando no texto pelo padrão BAAAAAAAAA.

- Suponha que combinamos 5 caracteres no padrão, com incompatibilidade no 6º caractere.
- Sabemos que os 6 caracteres anteriores no texto são BAAAAAB. 
- Não é necessário fazer backup da posição do texto! assuming { A, E



Knuth–Morris–Pratt algorithm. Método inteligente para sempre evitar backup. (!)

Autômato de estados finitos determinísticos (DFA)

DFA é uma máquina abstrata de busca de strings.

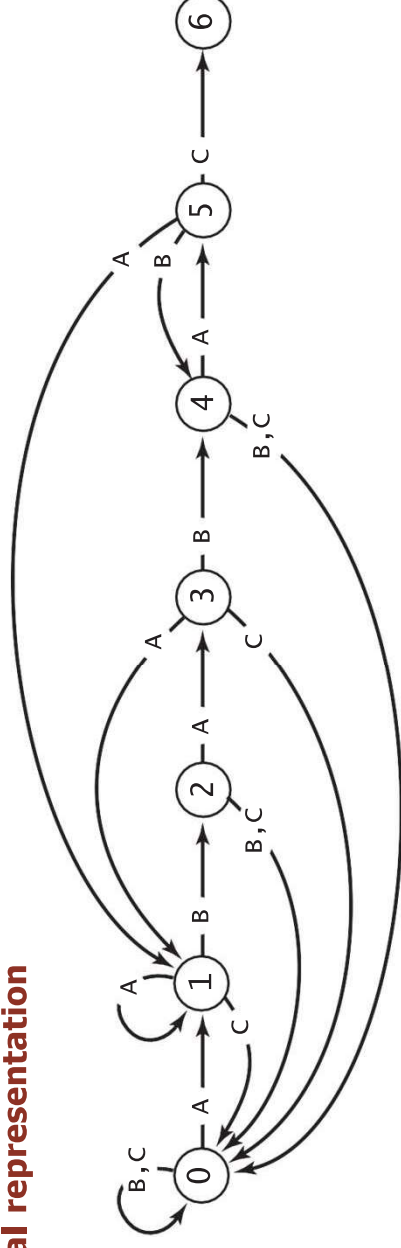
- Número finito de estados (incluindo início e parada).
- Exatamente uma transição para cada caractere no alfabeto.
- Aceite se a sequência de transições levar ao estado de parada.

internal representation

j	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A	1	1	3	1	5
	B	0	2	0	4	0
	C	0	0	0	0	6

If in state j reading char c :
if j is 6 halt and accept
else move to state $dfa[c][j]$

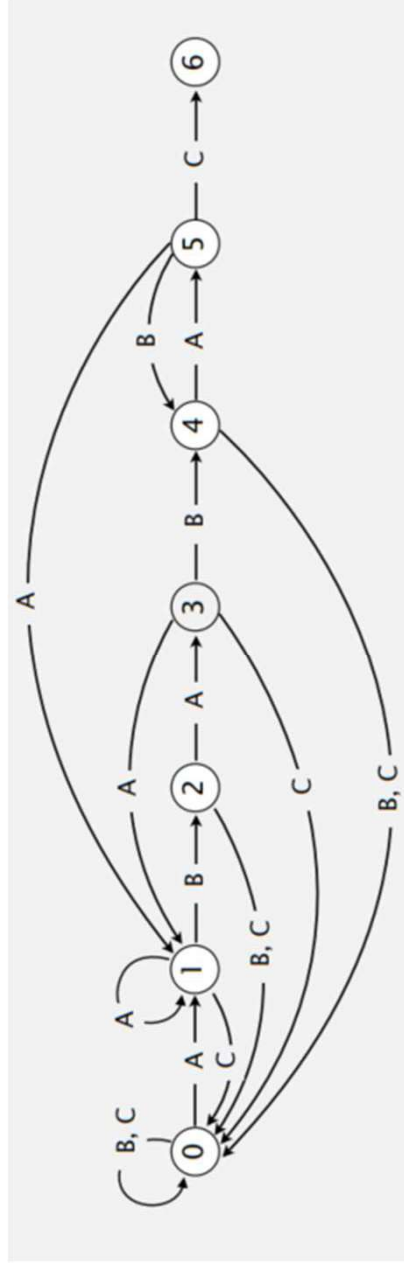
graphical representation



Knuth-Morris-Pratt demo: DFA

A A B A C A A B A B A C A A

	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A	1	1	3	1	5
	B	0	2	0	4	0
	C	0	0	0	0	6

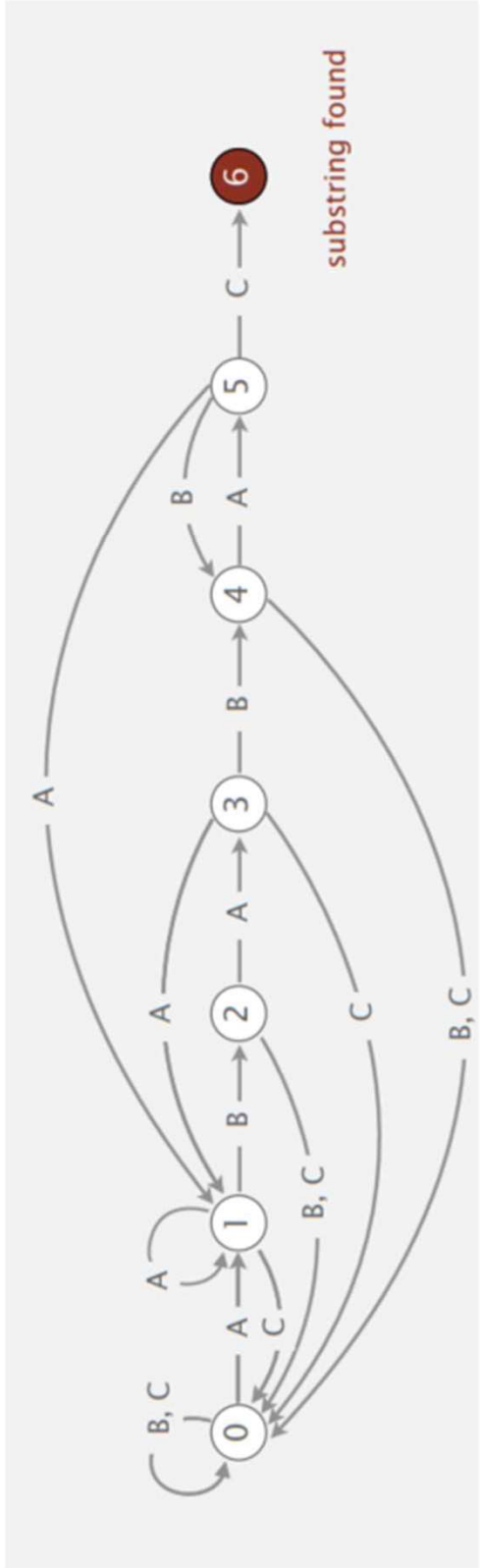


Knuth-Morris-Pratt demo: DFA

A A B A C A A B A B A C A A A

↑

	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A	1	3	1	5	1
B	0	2	0	4	0	4
C	0	0	0	0	0	6

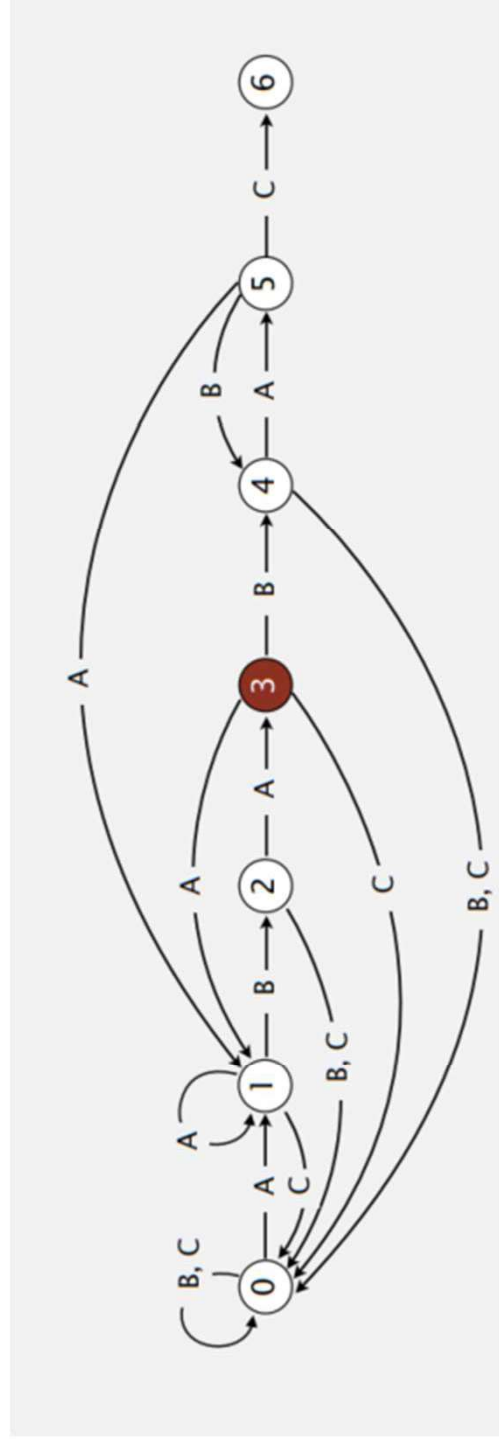


Interpretation of Knuth-Morris-Pratt DFA

- P. Qual é a interpretação do estado do DFA após a leitura em $\text{txt}[i]$?
- R. Estado = número de caracteres no padrão que foram correspondidos.
- Ex. DFA está no estado 3 após a leitura em $\text{txt}[0..6]$.

length of longest prefix of $\text{pat}[]$
that is a suffix of $\text{txt}[0..i]$

i								
0	1	2	3	4	5	6	7	8
$\text{txt} \longrightarrow$								
B	C	B	A	A	B	A	C	A
suffix of $\text{txt}[0..6]$								
$\text{pat} \longrightarrow$								
			0	1	2	3	4	5
			A	B	A	B	A	C
prefix of $\text{pat}[]$								



Knuth-Morris-Pratt substring search: Implementação em JAVA

Principais diferenças da implementação de força bruta.

- É necessário pré-calcular `dfa[][]` a partir do padrão.
- O índice de texto `i` nunca diminui.
- Pode usar input stream

```
public int search(String txt)
{
    int i, j, N = txt.length();
    for (i = 0, j = 0; i < N && j < M; i++)
        j = dfa[txt.charAt(i)][j];
    if (j == M) return i - M;
    else      return N;
}
```



Tempo de execução.

- Simular DFA em texto: no máximo N caracteres de acesso.
- Como Construir DFA de forma eficiente?

Knuth-Morris-Pratt substring search: Java implementation

Principais diferenças da implementação de força bruta.

- É necessário pré-calcular `dfa[][]` a partir do padrão.
- A posição de texto `i` nunca diminui.

```
public int search(In in)
{
    int i, j;
    for (i = 0, j = 0; !in.isEmpty() && j < M; i++)
        j = dfa[in.readChar()][j];
    if (j == M) return i - M;
    else      return NOT_FOUND;
}
```



no backup

Knuth-Morris-Pratt: Construção do DFA

Incluir um estado para cada caractere no padrão (mais o estado de aceitação).

	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A					
	B					
	C					

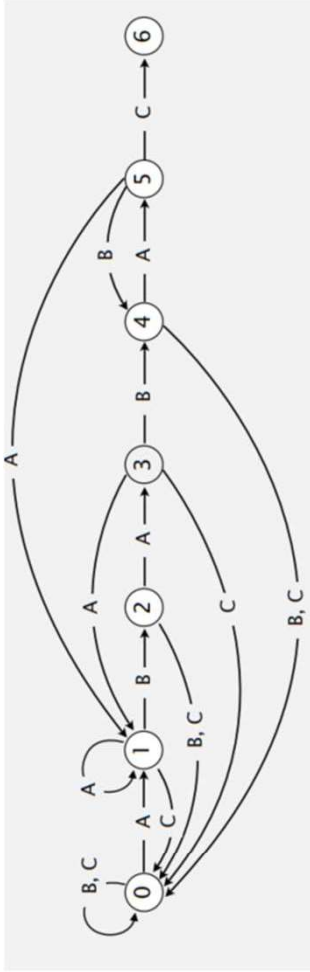
Construindo o DFA para pesquisa de substring KMP para ABABAC



Knuth-Morris-Pratt demo: Construção DFA

	0	1	2	3	4	5
pat.charAt (j)	A	B	A	B	A	C
dfa[][j]	A	1	3	1	5	1
B	0	2	0	4	0	4
C	0	0	0	0	0	6

Construindo o DFA para pesquisa de substring KMP para ABABAC



Como construir DFA a partir de padrões?

Incluir um estado para cada caractere no padrão (mais o estado de aceitação).

	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A					
	B					
	C					

0

1

2

3

4

5

6

Como construir DFA a partir de padrões?

Transição de Match. Se no estado j e o próximo char $c == \text{pat.charAt}(j)$, vá para $j+1$.

↑
first j characters of pattern
have already been matched

↑
next char matches

↑
now first $j+1$ characters of
pattern have been matched

	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
A	1		3		5	
B		2		4		
C						6



Como construir DFA a partir de padrões?

Falha de Transição. Se no estado j e próximo $\text{char } c \neq \text{pat.charAt}(j)$, então os últimos $j-1$ caracteres de entrada são $\text{pat}[1..j-1]$, seguidos por c .

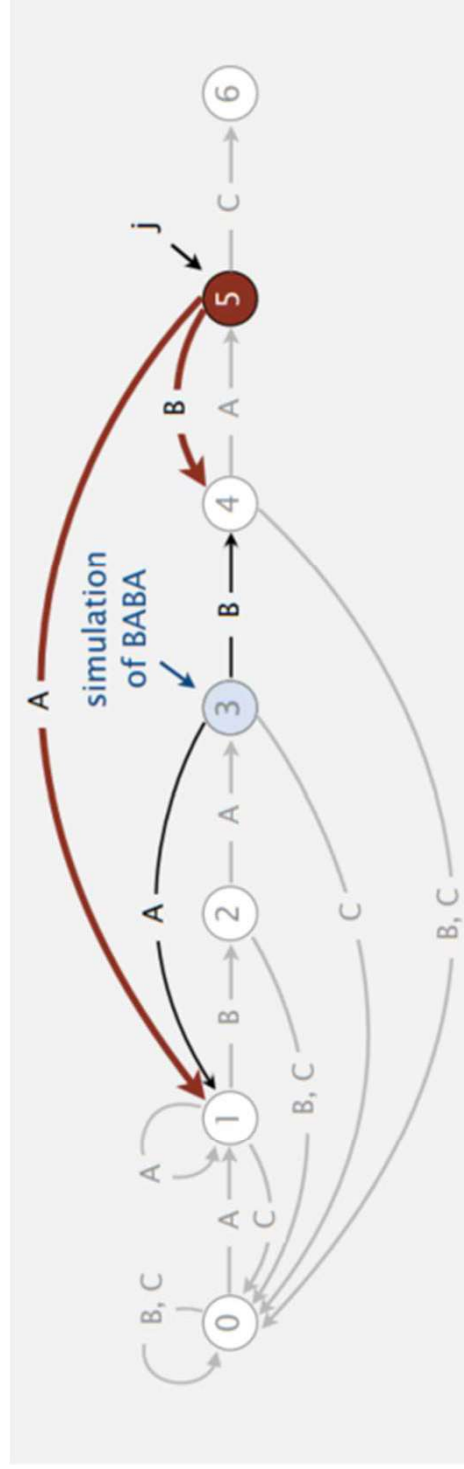
Para calcular $\text{dfa}[c][j]:::$ Simule $\text{pat}[1..j-1]$ on DFA e tome a transição c .

Tempo de execução. Parece exigir j passos.

↖
still under construction (!)

Ex. $\text{dfa}['A'][5] = 1;$ $\text{dfa}['B'][5] = 4$
simular BABA;
pegue a transição 'A'
= $\text{dfa}['A'][3]$

j	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C



Como construir DFA a partir de padrões?

Falha de Transição. Se no estado j e próximo char $c \neq \text{pat.charAt}(j)$, então os últimos $j-1$ caracteres de entrada são $\text{pat}[1..j-1]$, seguidos por c .

estado X

Para calcular $\text{dfa}[c][j] :: \text{Simule pat}[1..j-1]$ on DFA e tome a transição c .

. Running time. Leva apenas **tempo constante se mantivermos o estado X** .

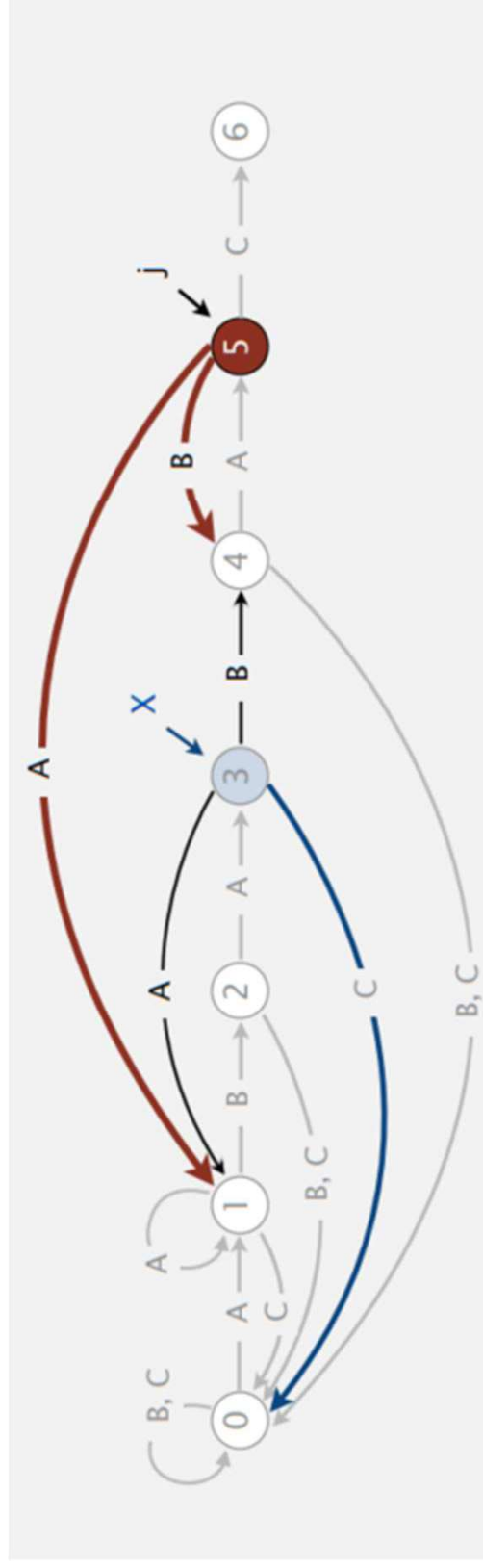
Ex. $\text{dfa}['A'][5] = 1;$ $\text{dfa}['B'][5] = 4$ $X' = 0$

estado X , pegue a
transição ' A '
 $= \text{dfa}['A'][X]$

estado X , pegue
a transição ' B '
 $= \text{dfa}['B'][X]$

estado X , pegue a
transição ' C '
 $= \text{dfa}['C'][X]$

0	1	2	3	4	5
A	B	A	B	A	C



Knuth-Morris-Pratt:

Construção do DFA em tempo linear

Incluir um estado para cada caractere no padrão (mais o estado de aceitação).

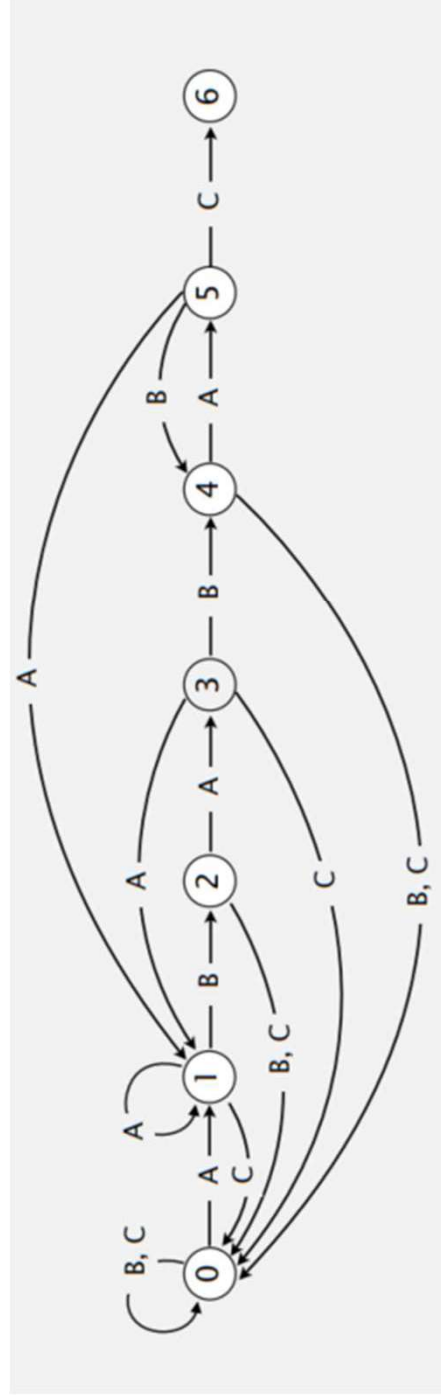
	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A	B	A	B	A	C

Construindo o DFA para pesquisa de substring KMP para ABABAC



	0	1	2	3	4	5
pat.charAt (j)	A	B	A	B	A	C
dfa[][j]	A	1	1	3	1	5
	B	0	2	0	4	4
	C	0	0	0	0	6

Construindo o DFA para pesquisa de substring KMP para ABABAC



Construção DFA para KMP substring search: implementação em Java

Para cada estado j

- Copie `dfa[][X]` para `dfa[][j]` para caso de falha.
- Atribua `dfa[pat.charAt(j)][j]` to $j+1$ para caso de match.
- Atualiza x .

```
public KMP(String pat)
{
    this.pat = pat;
    M = pat.length();
    dfa = new int[R][M];
    dfa[pat.charAt(0)][0] = 1;
    for (int x = 0, j = 1; j < M; j++)
    {
        for (int c = 0; c < R; c++)
            dfa[c][j] = dfa[c][x];
        dfa[pat.charAt(j)][j] = j+1;
        x = dfa[pat.charAt(j)][x];
    }
}
```

copy mismatch cases
set match case
update restart state

Tempo de execução.

M acessos de caracteres (mas espaço/tempo proporcional a $R \cdot M$).

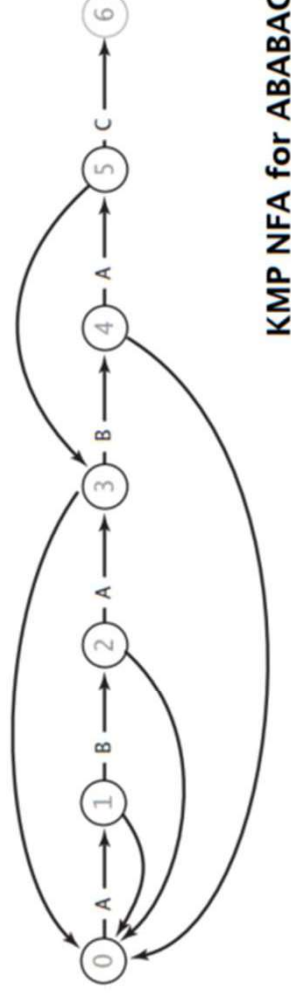
Análise de Pesquisa de Substring KMP

Primícia. A pesquisa de substring KMP acessa no máximo $M + N$ caracteres para procurar um padrão de comprimento M em um texto de comprimento N .

Obs. Cada caractere de padrão é acessado uma vez ao construir o DFA; cada caractere de texto acessado uma vez (no pior caso) ao simular o DFA.

Primícia. KMP constrói $dfa[][]$ em tempo e espaço proporcionais a $R M$.

Alfabetos maiores. Versão melhorada de KMP constrói $nfa[]$ em tempo e espaço proporcionais a M .



Knuth-Morris-Pratt: breve história

Descoberto independentemente por dois teóricos e um hacker

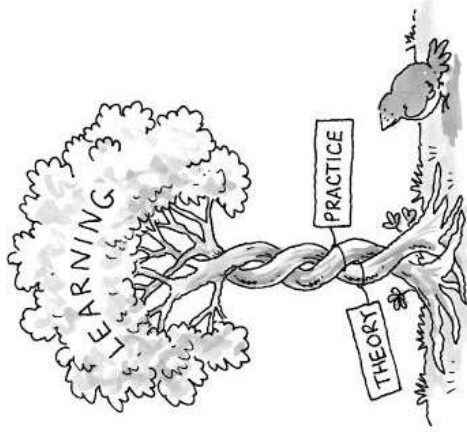
- Knuth: inspirado pelo teorema esotérico, descobriu o algoritmo linear.
- Pratt: tornou o tempo de execução independente do tamanho do alfabeto
- Morris: construiu um editor de texto para o computador CDC 6400
- A teoria encontra a prática

SIAM J. COMPUT.
Vol. 6, No. 2, June 1977

FAST PATTERN MATCHING IN STRINGS*

DONALD E. KNUTH[†], JAMES H. MORRIS, JR.[‡] AND VAUGHAN R. PRATT[¶]

Abstract. An algorithm is presented which finds all occurrences of one given string within another, in running time proportional to the sum of the lengths of the strings. The constant of proportionality is low enough to make this algorithm of practical use, and the procedure can also be extended to deal with some more general pattern-matching problems. A theoretical application of the algorithm shows that the set of concatenations of even palindromes, i.e., the language $\{\alpha\alpha^R\}^*$, can be recognized in linear time. Other algorithms which run even faster on the average are also considered.



Don Knuth



Jim Morris



Vaughan Pratt

Boyer-Moore



Robert Boyer



J. Strother Moore

Boyer-Moore: heurística de falha de caractere

Ideia Básica

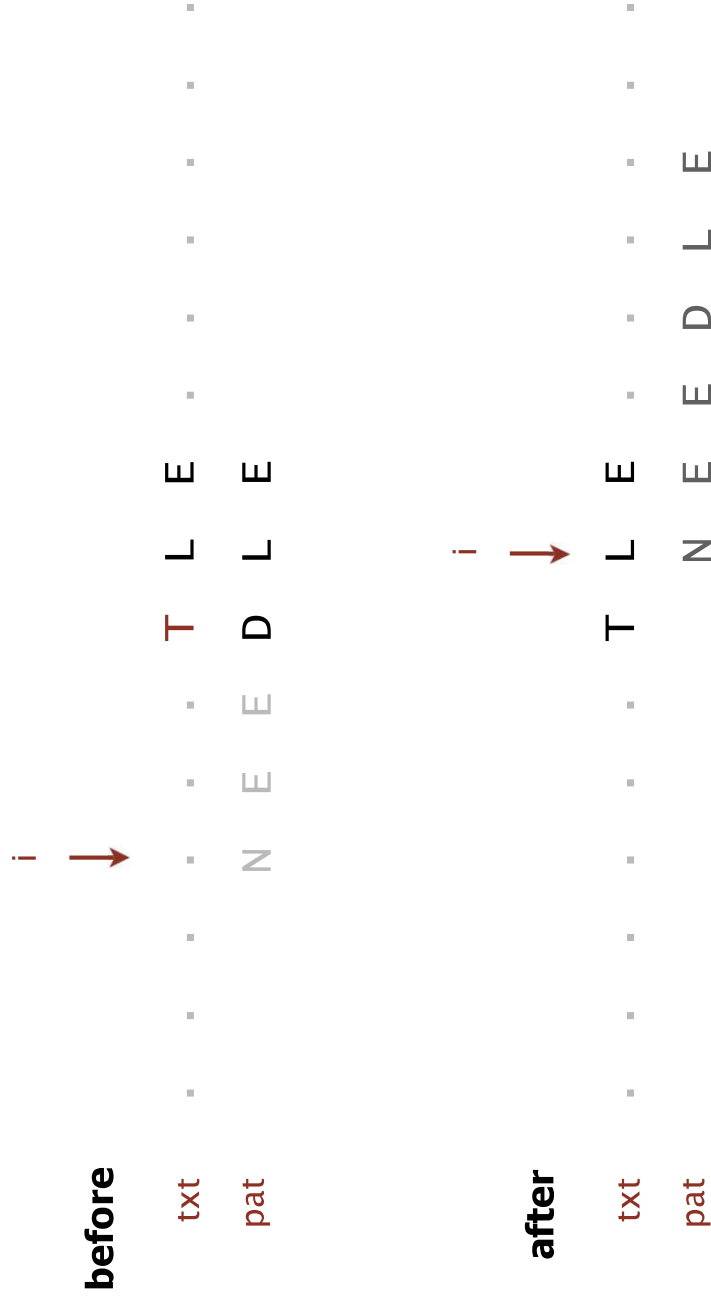
- Escaneie os caracteres em padrão da direita para a esquerda.
- É possível pular até M caracteres de texto ao encontrar um que não esteja no padrão.

i	j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
<i>text</i> →		F	I	N	D	I	N	A	H	A	Y	S	T	A	C	K	N	E	E	D	L	E	I	N	A
0	5	N	E	E	D	L	E	← <i>pattern</i>																	
5	5						N	E	E	D	L	E													
11	4												N	E	E	D	L	E							
15	0																N	E	E	D	L	E			
↙ <i>return i = 15</i>																									

Boyer-Moore: heurística de falha de caractere

Q. Quando pular?

Caso 1. O caractere de falha não está no padrão.

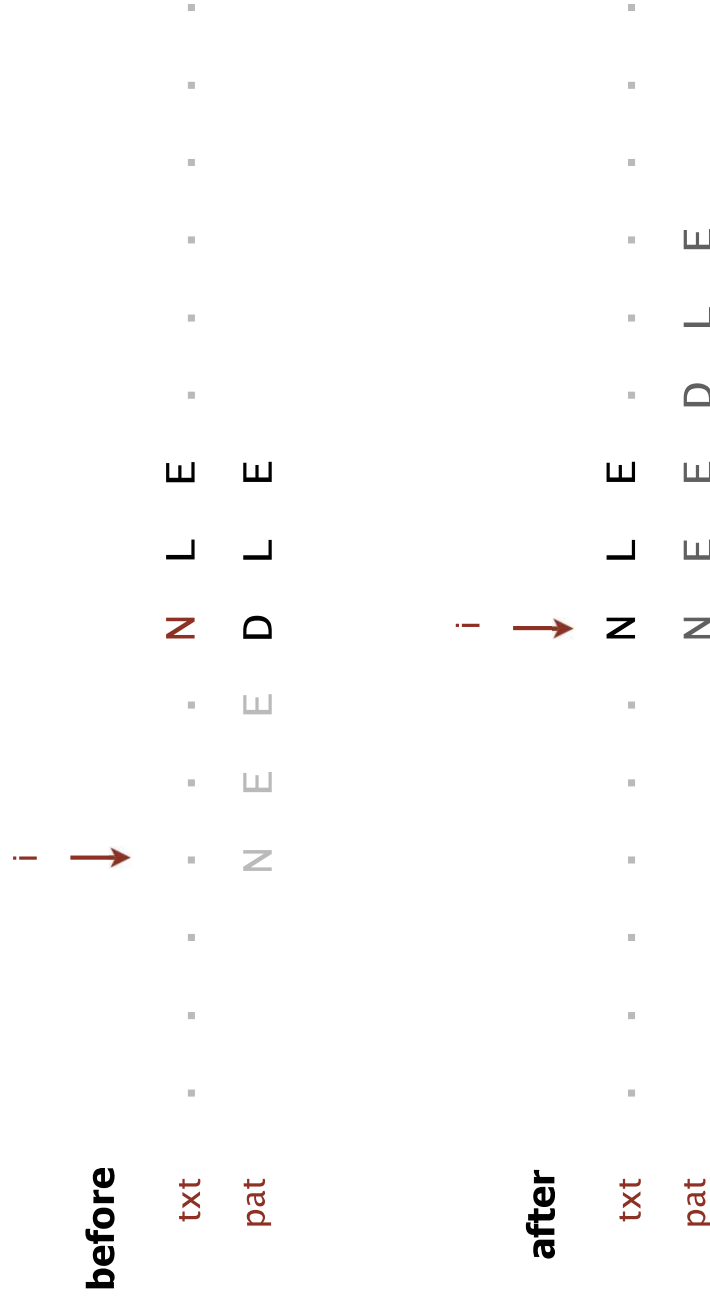


Falha no caractere 'T' não está no padrão: incrementa i após 'T'

Boyer-Moore: heurística de falha de caractere

Q. Quando publicar?

Caso 2a. Falha de padrão de caractere



caractere de falha 'N' no padrão: alinhar o texto 'N' com o padrão mais à direita 'N'

Boyer-Moore: heurística de falha de caractere

Q. Quando pular?

Case 2b. Falha de padrão de caractere. (mas a heurística não ajuda).

before

i ↓

txt E L E

pat N E E D L E

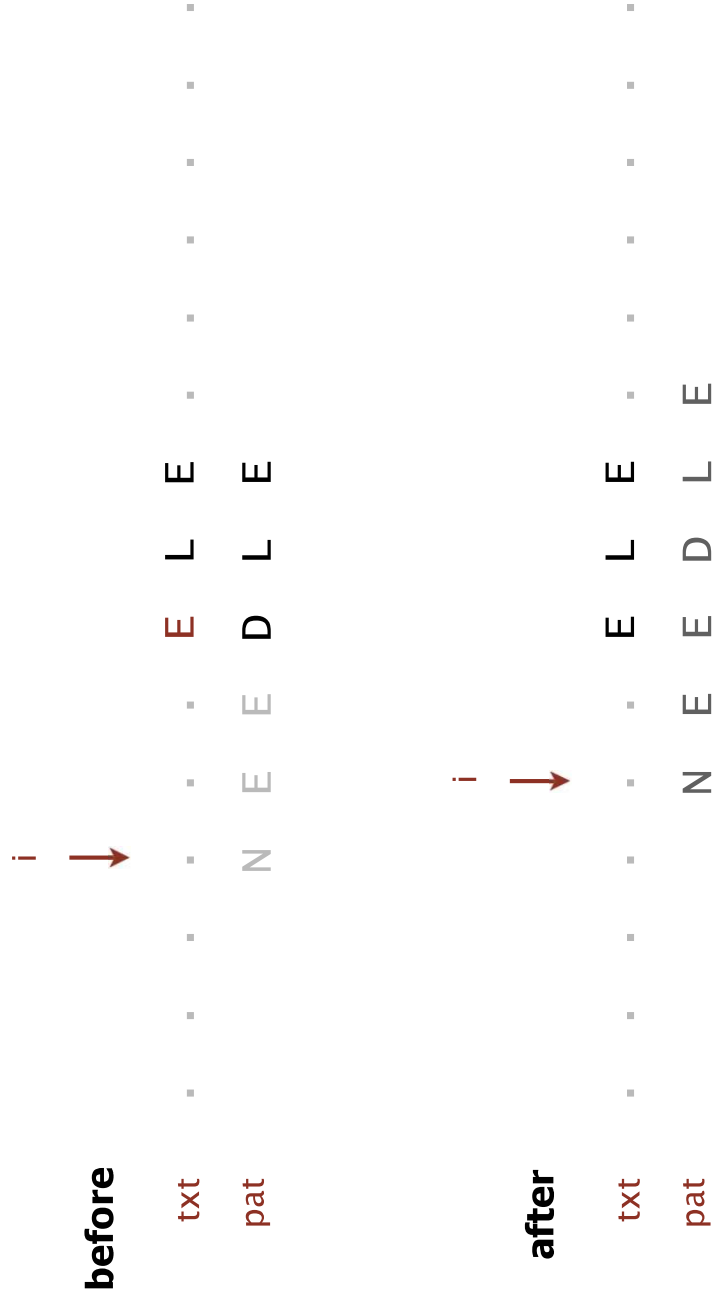
Alinhado com o E mais à direita? ↓

caractere incompatível 'E' no padrão: alinhar o texto 'E' com o padrão mais à direita 'E'?

Boyer-Moore: heurística de falha de caractere

Q. Quando pular?

Case 2b. Falha de padrão de caractere. (mas a heurística não ajuda).



Falha no caractere 'E' no padrão: incrementar i de 1

Boyer-Moore: heurística de falha de caractere

Q. Quando pular?

A. Pré-calcule o índice da ocorrência mais à direita do caractere c no padrão.

```
right = new int[R];
for (int c = 0; c < R; c++)
    right[c] = -1;
for (int j = 0; j < M; j++)
    right[pat.charAt(j)] = j;
```

<u>c</u>	N	E	E	D	L	E
	0	1	2	3	4	5
A	-1	-1	-1	-1	-1	-1
B	-1	-1	-1	-1	-1	-1
C	-1	-1	-1	-1	-1	-1
D	-1	-1	-1	3	3	3
E	-1	1	2	2	2	5
...						
L	-1	-1	-1	-1	4	4
M	-1	-1	-1	-1	-1	-1
N	-1	0	0	0	0	0
...						

right[c]

-1

-1

-1

3

5

-1

4

-1

0

-1

Boyer-Moore skip table computation

Boyer-Moore: Implementação em Java

```
public int search(String txt)
{
    int N = txt.length();
    int M = pat.length();
    int skip;
    for (int i = 0; i <= N-M; i += skip)
    {
        skip = 0;
        for (int j = M-1; j >= 0; j--)
        {
            if (pat.charAt(j) != txt.charAt(i+j))
            {
                skip = Math.max(1, j - right[txt.charAt(i+j)]);
                break;
            }
        }
        if (skip == 0) return i;
    }
    return N;
}
```

compute skip value

in case other term is nonpositive

Boyer-Moore: análise

Propriedade. A busca por substring com a heurística de caracteres incompatíveis de Boyer-Moore leva cerca de $\sim N / M$ caracteres em comparação à busca por um padrão de comprimento M em um texto de comprimento N .
sublinear!

Pior Caso. Pode ser tão ruim quanto $\sim MN$.

i	skip	0	1	2	3	4	5	6	7	8	9	
		txt → B B B B B B B B B B B										
0	0	A	B	B	B	B	← pat					
1	1		A	B	B	B	B					
2	1			A	B	B	B	B				
3	1				A	B	B	B	B			
4	1					A	B	B	B	B		
5	1						A	B	B	B	B	

Boyer-Moore variante. A busca por substring com a heurística de caracteres incompatíveis de Boyer-Moore leva cerca de $\sim N / M$ caracteres em comparação à busca por um padrão de comprimento M em um texto de comprimento N .

Rabin-Karp



Michael Rabin
Dick Karp

Rabin-Karp pesquisa digital

Ideia básica = hash modular.

- Calcule o hash do $\text{pat}[0..M-1]$.
- para cada i , calcule um hash de $\text{txt}[i..M+i-1]$.
- If pattern hash = text substring hash, verifique se é um match.

pat.charAt(i)																				
i	0	1	2	3	4													997 = 613		
		txt.charAt(i)																		
i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15				
	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3				
0	3	1	4	1	5	5	%	997	=	508										
1	1	4	1	5	9	%	997	=	201											
2	4	1	5	9	2	%	997	=	715											
3	1	5	9	2	6	%	997	=	971											
4	5	9	2	6	5	%	997	=	442											
5	9	2	6	5	3	%	997	=	929											
6	← return i = 6	2	6	5	3	5	%	997	=	613								match ↗		

match



modular hashing with $R = 10$ and $\text{hash}(s) = s \pmod{997}$

Aritmética Modular

Truque matemático. Para manter os números pequenos, pegue resultados intermediários módulo Q .

$$\begin{aligned}\text{Ex.} \quad & (10000 + 535) * 1000 \pmod{997} \\ &= (30 + 535) * 3 \pmod{997} \\ &= 1695 \pmod{997} \\ &= 698 \pmod{997}\end{aligned}$$

$$(a + b) \bmod Q = ((a \bmod Q) + (b \bmod Q)) \bmod Q$$

$$(a * b) \bmod Q = ((a \bmod Q) * (b \bmod Q)) \bmod Q$$

Calculo eficiente da função hash

Usando a função hash modular. Usando a notação t_i para `txt.charAt(i)`, Calculamos

$$x_i = t_i R^{M-1} + t_{i+1} R^{M-2} + \dots + t_{i+M-1} R^0 \pmod{Q}$$

Sendo: M -dígito, inteiro de base R , módulo Q .

Método de Horner. Método de tempo linear para avaliar polinômio de grau M .

		pat.charAt()				
i	0	1	2	3	4	
	2	6	5	3	5	
0	2	% 997	= 2			
1	2	6	% 997 = (2*10 + 6) % 997	= 26		
2	2	6	5	% 997 = (26*10 + 5) % 997	= 265	
3	2	6	5	3	% 997 = (265*10 + 3) % 997	= 659
4	2	6	5	3	5	% 997 = (659*10 + 5) % 997 = 613

```
// Compute hash for M-digit key
private long hash(String key, int M)
{
    long h = 0;
    for (int j = 0; j < M; j++)
        h = (h * R + key.charAt(j)) % Q;
    return h;
}
```

$$\begin{aligned} 26535 &= 2*10000 + 6*1000 + 5*100 + 3*10 + 5 \\ &= (((((2) * 10 + 6) * 10 + 5) * 10 + 3) * 10 + 5) \end{aligned}$$

Calculo eficiente da função hash

Desafío.

$$x_i = t_i R^{M-1} + t_{i+1} R^{M-2} + \dots + t_{i+M-1} R^0 x_{i+1} =$$

$$t_{i+1}R^{M-1} + t_{i+2}R^{M-2} + \dots + t_{i+M}R^0$$

Propriedade chave. Pode atualizar a função hash "rolling" em tempo constante!

$$x_{i+1} = (x_i - t_i R^{M-1}) R + t_{i+M}$$

current value subtract leading digit multiply by radix add new trailing digit

(can precompute R^{M-1})

[illegible]

Rabin-Karp substring search

Primeiras entradas: Use a regra de Horner.

Entrada restantes: Use hash rolling (para evita overflow).

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
0	3	% 997	=	3												
1	3	1	% 997	=	(3*10 + 1)	% 997	=	31								
2	3	1	4	% 997	=	(31*10 + 4)	% 997	=	314							
3	3	1	4	1	% 997	=	(314*10 + 1)	% 997	=	150						
4	3	1	4	1	5	% 997	=	(150*10 + 5)	% 997	=	508					
5	1	4	1	5	9	% 997	=	((508 + 3*(997 - 30))*10 + 9)	% 997	=	201					
6	4	1	5	9	2	% 997	=	((201 + 1*(997 - 30))*10 + 2)	% 997	=	715					
7	1	5	9	2	6	% 997	=	((715 + 4*(997 - 30))*10 + 6)	% 997	=	971					
8	5	9	2	6	5	% 997	=	((971 + 1*(997 - 30))*10 + 5)	% 997	=	442					
9	9	2	6	5	3	% 997	=	((442 + 5*(997 - 30))*10 + 3)	% 997	=	929					
10	← return i-M+1 = 6	2	6	5	3	5	% 997	=	((929 + 9*(997 - 30))*10 + 5)	% 997	=	613				

Horner's rule

rolling hash

match

$$-30 \pmod{997} = 997 - 30$$

$$10000 \pmod{997} = 30$$

Rabin-Karp: implementação em Java

```
public class RabinKarp
{
    private long patHash;    // pattern hash value
    private int M;           // pattern length
    private long Q;          // modulus
    private int R;           // radix
    private long RM1;        //  $R^{M-1} \% Q$ 

    public RabinKarp(String pat) {
        M = pat.length();
        R = 256;
        Q = LongRandomPrime();
    }

    RM1 = 1;
    for (int i = 1; i <= M-1; i++)
        RM1 = (R * RM1) % Q;
    patHash = hash(pat, M);
}

private long hash(String key, int M)
{
    /* as before */
}

public int search(String txt)
{
    /* see next slide */
}
```

a large prime
(but avoid overflow)

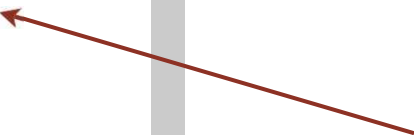
precompute $R^{M-1} \pmod{Q}$

Rabin-Karp: Java implementação (continuação)

Versão Monte Carlo. Retorna o índice se haver match.

```
public int search(String txt)
{
    int N = txt.length();
    int txtHash = hash(txt, M);
    if (pathHash == txtHash) return 0;
    for (int i = M; i < N; i++)
    {
        txtHash = (txtHash + Q - RM*txt.charAt(i-M) % Q) % Q;
        txtHash = (txtHash*R + txt.charAt(i)) % Q;
        if (pathHash == txtHash) return i - M + 1;
    }
    return N;
}
```

check for hash collision
using rolling hash function



Versão Las Vegas. Verificar se há correspondência de *hash*; continuar se colisão for falsa.

Análise Rabin-Karp

Teoria. se Q for um primo aleatório suficientemente grande (cerca MN^2), então a probabilidade de uma colisão falsa $1 / N$.

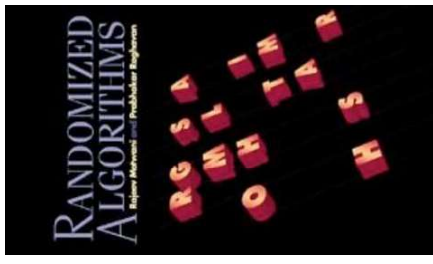
Prática. Escolha Q para ser um primo grande (mas não tão grande para causar *overflow*). Sob suposições razoáveis, a probabilidade de uma colisão é de cerca de $1 / Q$.

Versão Monte Carlo.

- Sempre roda em tempo linear.
- Extremamente provável que retorne a resposta correta (mas nem sempre!).

Versão Las Vegas .

- Sempre retorna a resposta correta.
- Extremamente provável que seja executado em tempo linear (pior caso MN).



Pesquisa digital de Rabin-Karp

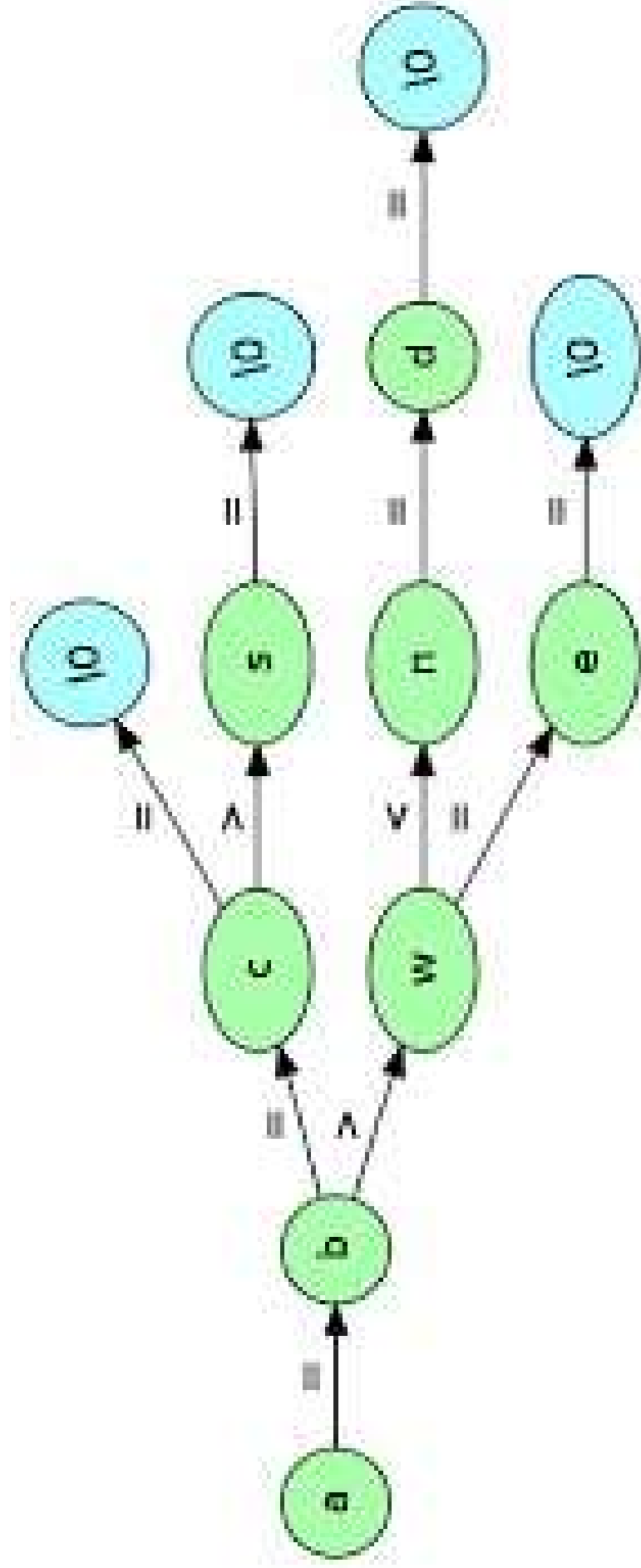
Vantagens:

- estende-se a padrões 2d.
- estende-se para encontrar múltiplos padrões.

Desvantagens:

- Operações aritméticas mais lentas do que char compara.
- A versão de Las Vegas requer *backup*.
- Garantia ruim no pior cenário.

Árvore Trie



Substring search cost summary

Cost of searching for an M -character pattern in an N -character text.

algorithm	version	operation count		backup in input?	correct?	extra space
		guarantee	typical			
brute force	—	MN	$1.1N$	yes	yes	1
Knuth-Morris-Pratt	full DFA (Algorithm 5.6)	$2N$	$1.1N$	no	yes	MR
	mismatch transitions only	$3N$	$1.1N$	no	yes	M
	full algorithm	$3N$	N/M	yes	yes	R
Boyer-Moore	mismatched char heuristic only (Algorithm 5.7)	MN	N/M	yes	yes	R
	Monte Carlo (Algorithm 5.8)	$7N$	$7N$	no	yes [†]	1
Rabin-Karp [†]	Las Vegas	$7N$ [†]	$7N$	yes	yes	1

[†] probabilistic guarantee, with uniform hash function

Referências

- K. Wayne and R. Sedgewick, Algorithms, Princeton University, Coursera online course, <https://algs4.cs.princeton.edu/home/>